

Automating Compiler Flags Selection for SW Processors via Small Language Models

Chaowei Zhao

Zhengzhou University

Jinyang Yao

PLA Information Engineering University

Bo Zhao

PLA Information Engineering University

Lili Liu

PLA Information Engineering University

Haoran Wang

PLA Information Engineering University

Yadian Cheng

PLA Information Engineering University

Ping Zhang

PLA Information Engineering University

Jinlong Xu

PLA Information Engineering University

Kai Nie

`ieknie@zzu.edu.cn`

Zhengzhou University

Research Article

Keywords: Compiler Optimization, Compiler Flags, Large Language Models

Posted Date: June 8th, 2026

DOI: <https://doi.org/10.21203/rs.3.rs-9831666/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Automating Compiler Flags Selection for SW Processors via Small Language Models

Chaowei Zhao¹[0009-0008-1933-2423]^{*}, Jinyang Yao^{2*}, Bo Zhao², Lili Liu²,
Haoran Wang², Yadian Cheng², Ping Zhang², Jinlong Xu², and Kai Nie¹ 

¹ ZhengZhou University, ZhengZhou, China
zhaocw@gs.zzu.edu.cn, ieknie@zzu.edu.cn

² Information Engineering University, ZhengZhou, China
yaojy1024@126.com, zhaob07@tsinghua.org.cn, liull_lili@163.com,
wang_hr0224@163.com, chengyd1025@163.com, 13674910726@139.com,
longkaizh@126.com

Abstract. Finding effective compiler flag combinations for programs under constrained resources is a fundamental challenge in compiler optimization. The problem is inherently NP-hard, requiring a search within an extremely high-dimensional discrete space. While traditional heuristic search methods can be effective, they are often computationally expensive. We propose a machine learning framework, **AutoCFS-SLMs**, integrated into the industrial-strength GCC compiler, to predict near-optimal compiler flag combinations for generating high-performance executables on SW Processors. To the best of our knowledge, this is the first work to integrate Small Language Models (SLMs)—typically ranging from hundreds of millions to a few billion parameters—into an industrial compiler for performance optimization. By transforming the costly search problem into an efficient inference task, our approach is compared against the current state-of-the-art `-O3` optimization level in GCC using the SPEC CPU 2017 integer speed benchmark. Experimental results on a test set independent of the training data show that our method achieves an average performance improvement of 7.6% compared to `-O3`. This study demonstrates that pre-trained SLMs can effectively capture expert optimization knowledge, offering a practical and efficient solution for low-cost, automated performance tuning in industrial compilers.

Keywords: Compiler Optimization · Compiler Flags · Large Language Models

1 Introduction

Compiler flag optimization is fundamentally a search for the global optimum within an ultra-high-dimensional, discrete, and non-linear interaction space. Formally, this task can be defined as follows: given a source code **P**, a search algorithm or machine learning model aims to produce a combination of compiler

^{*} Both authors contributed equally to this research.  Corresponding author

flags **C** such that the executable compiled with **C** achieves optimal runtime performance on a target architecture.

Industrial compilers generally employ fixed, program-agnostic compiler flag combinations, such as the optimization level `-O2` and `-O3`. While these presets generate code of reasonable average quality across broad benchmark suites, they inevitably miss optimization opportunities specific to individual programs. Consequently, such one-size-fits-all combinations fail to deliver optimal performance for most applications.

Previous research applying machine learning to compiler optimization has established a clear consensus: traditional, fixed optimization combinations struggle to adapt to diverse program characteristics, whereas learning-based methods can achieve significant performance gains through adaptive decision-making. Despite notable progress across various levels, several critical limitations persist.

Early work focused primarily on iterative compilation-based search algorithms. These methods were constrained by the high cost of iterative compilation and the cold-start problem [1]. While auto-tuning frameworks like OpenTuner offer flexibility, they still rely on iterative compilation, resulting in low search efficiency and a dependence on carefully crafted, user-defined search spaces [2]. Subsequent research shifted towards data-driven machine learning methods, which reduced the need for expert knowledge but were often limited by the quality of manually engineered features [3] [4], leading to poor generalization.

In recent years, the proliferation of novel instruction set architectures, such as RISC-V and SW64, and the growing complexity of heterogeneous computing have exacerbated the limitations of traditional methods—namely, low efficiency and poor adaptability—making their integration into industrial compilers challenging. Following the revolutionary advances of Large Language Models (LLMs) in code understanding and logical reasoning, researchers have begun exploring their application to compiler optimization, hoping they can comprehend source code semantics and reason about corresponding compilation strategies [5] [6], or even translate high-level code directly to assembly [7] [8] [9]. However, LLMs face significant constraints, including context length limits, insufficient mathematical reasoning capability, high inference latency, and enormous data requirements. For the downstream task of compiler flag combination optimization, training data must be collected through iterative compilation and execution on real hardware, making large-scale dataset creation prohibitively expensive.

In summary, traditional iterative compilation is effective but costly; conventional machine learning methods are limited by feature engineering and generalization; and LLM-based approaches demand vast resources [10]. In this context, SLMs present a promising new direction for tackling this high-dimensional optimization problem. This study explores the use of SLMs for compiler flag combination optimization and their integration into the industrial GCC compiler. A central question is: what are the effective boundaries of SLMs when applied to this combinatorial explosion problem? Specifically, can SLM-recommended flag combinations consistently outperform baseline optimization levels across programs of varying complexity and characteristics, and how do they compare to

the performance of larger LLMs? Through systematic benchmarking, this work aims to precisely delineate the scope and limits of the SLM-based approach.

The main contributions of this paper are as follows:

- We propose a lightweight, LLM-based compilation optimization method tailored for specific instruction set architectures, such as the SW64 processor used in our experiments. While prior LLM-based research shows promise, it often suffers from large model sizes, high inference overhead, massive data requirements, and insufficient adaptation to target hardware. Our approach innovatively adopts SLMs and tightly couples them with the compilation toolchain of the target processor. This ensures optimization effectiveness while drastically reducing the computational and deployment costs, offering a practical pathway for applying language models in resource-constrained environments.
- We introduce a compilation safety backtracking mechanism. LLM-based flag optimization models can cause compilation failures by generating overly aggressive or incompatible flags, undermining its practical practicality. Our framework incorporates a dynamic backtracking mechanism at the compiler interface. When the model’s predicted flags lead to a compilation error, the mechanism automatically disables high-risk flags iteratively until compilation succeeds or falls back to a stable baseline optimization level, ensuring robustness and enhancing the method’s practical utility.
- We construct a performance-oriented dataset using real hardware and validate our method’s effectiveness and generalizability on standard benchmarks. The training programs are sourced from public code repositories, such as CodeNet and ExeBench, and augmented via LLM-generated code, filtered for successful compilation and execution. An adaptive genetic algorithm searches for near-optimal compiler flag combinations under limited computational budgets on real hardware, forming high-quality $\langle \textit{Prompt}, \textit{Label} \rangle$ pairs. This work provides a curated, domain-specific dataset for learning-based compilation optimization.

The remainder of this paper is organized as follows. Section 2 reviews related work on compiler flag combination optimization based on search, machine learning, and LLMs. Section 3 details the AutoCFS-SLMs framework. Section 4 elaborates on SLM pre-training and fine-tuning. Section 5 describes the backtracking mechanism and compiler integration. Section 6 presents experimental results. Section 7 summarizes the paper’s contributions.

2 Related Work

Compiler flag optimization has evolved through three paradigms: search-based iterative compilation, traditional machine learning, and LLM-based methods.

Search-Based Iterative Compilation. Early work [1] [11] showed that genetic algorithms can discover flag combinations surpassing expert-tuned levels.

While general and black-box, each candidate requires full compilation and execution. Triantafyllis et al. [1] reduced overhead via tree-structured search and static estimation, but reliance on pre-selected flags and inaccurate estimates limited generalization. Almagor et al. [11] revealed a landscape with many local optima, confirming feasibility but also highlighting the **one-search, one-use** dilemma—optimization knowledge is not transferable. This high cost motivated learning-based approaches.

Machine Learning with Hand-Crafted Features. To reduce search cost, subsequent work [12] used manually designed features from source code or IR to predict optimization strategies, enabling knowledge transfer across programs. However, feature engineering is labor-intensive and often fails to capture global semantics or subtle performance-critical patterns. Minor syntactic variations can cause large feature changes while semantically equivalent code may yield different features, capping representational power and generalization.

LLM-Based Methods. LLMs leverage large-scale pre-training to learn rich code representations, eliminating manual feature engineering. Recent studies [10] [13] [14] frame compiler optimization as sequence-to-sequence tasks, showing strong potential in understanding code intent and identifying optimization patterns [15] [16]. However, billion-parameter models incur high inference overhead and memory footprint, complicating integration into resource-constrained compiler workflows. Cloud-based deployment also raises data privacy concerns for proprietary code.

Summary. LLM-based methods represent the state of the art, but their practicality in resource-sensitive environments remains limited. This gap motivates our exploration of SLMs, which retain semantic understanding while being lightweight enough for seamless compiler integration.

3 Framework

This section details the architecture and workflow of the AutoCFS-SLMs framework. The objective is to leverage SLMs to automatically infer effective compiler flag combinations, thereby generating high-performance executables for target processors. The overall architecture is illustrated in Figure 1. The workflow consists of three main stages: (1) IR generation, where source code is transformed into Gimple IR; (2) Flag combination inference, where an SLM analyzes the IR and predicts an optimization combination; and (3) Robust compilation, where the compiler attempts to generate the executable using the predicted flags, with an automatic fallback mechanism in case of failure.

Gimple IR Generator. This component transforms the input source code into Gimple IR, a low-level, architecture-agnostic representation used within the GCC compiler. Prior work suggests that Gimple IR, by abstracting away syntactic details and presenting a standardized structure, can be more amenable to static analysis than raw source code [15]. This IR captures the essential control and data flow of the program, providing a consistent and informative input feature for the subsequent SLM.

SLMs. The SLM serves as the core inference engine of the framework. It takes the Gimble IR sequence as input and outputs a predicted combination of compiler flags. The model is designed to learn the mapping between code patterns and beneficial optimization strategies from a curated dataset of $\langle \textit{Gimble IR}, \textit{Compiler Flags} \rangle$ pairs. By employing a model with a parameter count in the range of several million, we maintain substantial representational capacity while ensuring the model is lightweight enough for practical integration and low-latency inference in resource-constrained compilation environments.

Compiler with Safety Backtracking. The GCC compiler is invoked with the flag combination recommended by the SLM. To ensure robustness, we integrate a *safety backtracking mechanism*. If the initial flag combination triggers a compilation error, this mechanism is activated. It automatically identifies and disables high-risk flags from the combination and re-attempts compilation. This process iterates until compilation succeeds or a fallback to a stable baseline such as the optimization level `-O2` is enforced. This subsystem guarantees that the framework always produces a valid executable, enhancing its practicality for automated deployment.

In summary, the AutoCFS-SLMs framework tightly integrates an IR-based code representation, a lightweight SLM for pattern recognition and decision-making, and a robust compiler interface with automatic error recovery. This integrated design automates the compiler flag selection process, aiming to improve performance while maintaining reliability, and provides a foundation for empirical evaluation.

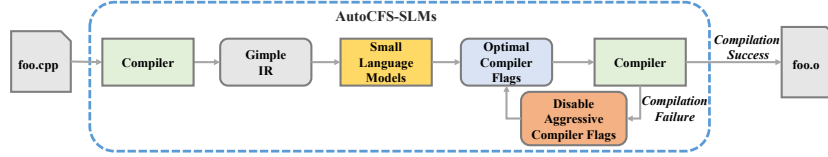


Fig. 1. Overview of the AutoCFS-SLMs framework. Source code `foo.cpp` is first converted to Gimble IR. An SLM analyzes the IR and recommends a compiler flag combination. The compiler uses these flags for compilation. Successful compilation yields an optimized object file `foo.o`. If compilation fails, the safety backtracking mechanism adjusts the flags, and the process retries, forming a robust closed loop.

4 Specializing and Training SLMs for Compiler Flag Optimization

This section details the two-stage training methodology for specializing SLMs to the compiler flag combination optimization task. We employ a pipeline of domain-specific pre-training followed by supervised fine-tuning. Section 4.1 describes the pre-training on a GCC Gimble IR corpus to adapt the model to the compiler domain. Section 4.2 presents the construction of the target task dataset and the subsequent fine-tuning process.

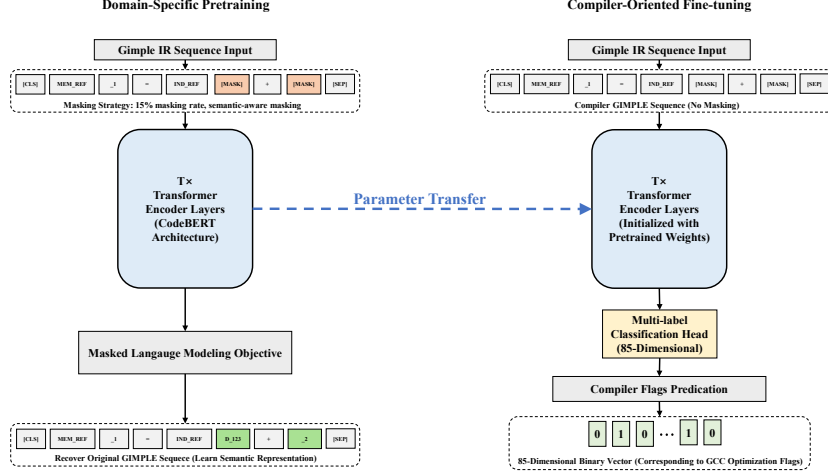


Fig. 2. The two-stage training paradigm for compiler flag combination optimization. The left side illustrates domain-specific pre-training on a large-scale GCC Gimple IR corpus using masked language modeling. The right side shows task-specific fine-tuning: the pre-trained model parameters are loaded and fine-tuned on a labeled dataset of $\langle \text{Gimple IR}, \text{Compiler Flags} \rangle$ pairs. These optimal flags are near-optimal combinations obtained via iterative compilation. A multi-label classification head is added to predict an 85-dimensional binary vector representing the activation of GCC optimization flags. The workflow transitions from learning general IR semantics to predicting compiler flags.

4.1 Domain-specific Pretraining on Compiler IRs

We formulate compiler flag optimization as a multi-label classification task, given a program’s Gimple IR, the model predicts an 85-bit vector indicating which flags to enable. This reduces the search space from the permutation space of flag orders to the power set of flags (2^{85}), making the problem tractable.

As shown in Figure 2, we adopt a two-stage training paradigm based on CodeBERT [17], a 110M-parameter model pre-trained on code. While CodeBERT excels at source code understanding, its comprehension of low-level compiler IR is limited because Gimple IR was absent from its pre-training corpus and differs significantly from source language syntax. To bridge this gap, we perform continued pre-training on a large-scale Gimple IR corpus.

Corpus Construction. We build a diverse Gimple IR corpus totaling 7.14 million lines, assembled from: (1) open-source benchmarks (CodeNet, ExeBench), (2) LLM-generated code compiled to IR, and (3) multiple optimization variants (-O0 to -O3) of each program. This ensures broad coverage of program structures and optimization states.

Vocabulary Expansion and Pre-training. Gimple IR introduces domain-specific tokens (e.g., SSA variables like `_1`, IR opcodes like `MEM_REF`). We expand the tokenizer vocabulary by adding approximately 200 such tokens to reduce out-

of-vocabulary rates. We then continue pre-training using masked language modeling with 15% masking probability, applying strategic masking that prioritizes semantically critical elements (variable references, operation types, data-flow tokens). The model is trained for 50,000 steps with batch size 128, using AdamW (learning rate 5×10^{-5}) and a linear warmup. This equips the model with specialized knowledge of compiler IR while preserving its general code understanding.

4.2 Task-Specific Fine-tuning

Adaptive Genetic Algorithm for Dataset Construction Training an effective SLM requires a large corpus of $\langle \text{Gimple IR}, \text{compiler flags} \rangle$ pairs. However, exhaustively exploring the 2^{85} possible flag combinations is infeasible. To efficiently collect high-performance configurations, we develop an automated pipeline based on an adaptive genetic algorithm (GA). Starting from the -O3 baseline, we manually select 85 additional flags with significant optimization potential. The GA then searches for flag combinations that outperform -O3, as illustrated in Figure 3.

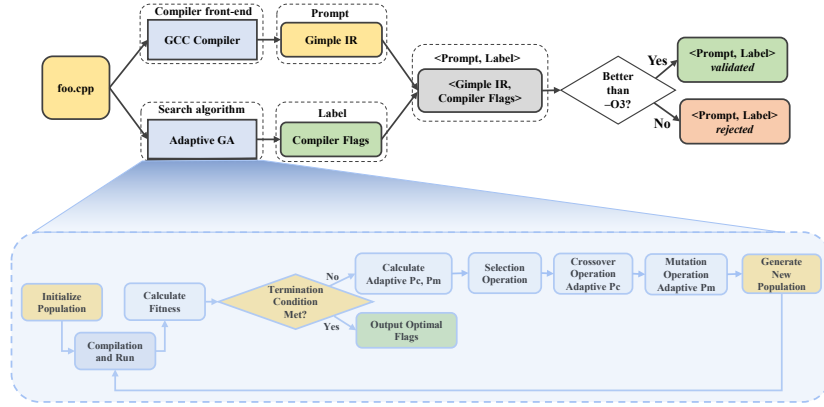


Fig. 3. Dataset construction workflow. For each program, Gimple IR is extracted, and an adaptive GA searches for high-performance flag combinations. Validated pairs that outperform -O3 are retained.

Each flag combination is encoded as an 85-bit binary chromosome. Fitness is defined as the speedup over -O3: $F(C_i) = T(-O3)/T(C_i)$, measured after three stable executions on dedicated CPU cores. The GA employs tournament selection ($k = 3$), single-point crossover, and uniform mutation. Crossover and mutation probabilities are dynamically adapted based on population diversity and convergence: high-fitness individuals receive lower mutation rates to preserve good genes, while low-fitness individuals receive higher rates to encourage exploration. If fitness stagnates, both probabilities increase to escape local optima.

We retain all evaluated configurations that surpass -O3, yielding over 100k $\langle \text{Gimple IR}, \text{flags} \rangle$ pairs from approximately 5,000 programs. Each pair consists

of the Gimple IR as input X and an 85-bit binary vector Y indicating enabled flags. This dataset serves as the foundation for supervised fine-tuning.

Supervised Fine-tuning for Compiler Flag Prediction We formulate compiler flag prediction as a multi-label classification task. Given a Gimple IR sequence G , the model predicts an 85-dimensional binary vector $\mathbf{y}$, where each entry indicates whether a specific flag should be enabled. We model the conditional probability as $P_\theta(\mathbf{y}|G) = \prod_{i=1}^{85} P_\theta(y_i|G)$, assuming conditional independence for tractability.

Model Adaptation. We use CodeBERT with 110M parameters as the base model, replacing its output head with a linear layer that projects the [CLS] token representation $\mathbf{h}_{[\text{CLS}]}$ to 85 logits, followed by sigmoid activation:

$$\hat{y}_i = \sigma(\mathbf{W}\mathbf{h}_{[\text{CLS}]} + \mathbf{b})_i.$$

Loss Function. To handle flag imbalance, we employ a weighted binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{k=1}^N \sum_{i=1}^{85} w_i [y_{ki} \log \hat{y}_{ki} + (1 - y_{ki}) \log(1 - \hat{y}_{ki})],$$

with weights $w_i \propto 1/\sqrt{p_i + \epsilon}$, where p_i is the empirical frequency of flag i .

Parameter-Efficient Fine-Tuning. We use Low-Rank Adaptation (LoRA) to reduce trainable parameters. For each query/value projection $\mathbf{W}_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, the forward pass becomes:

$$\mathbf{h} = \mathbf{W}_0 \mathbf{x} + \frac{\alpha}{r} \mathbf{B} \mathbf{A} \mathbf{x},$$

with rank $r = 8$, scaling $\alpha = 16$, and $\mathbf{A}, \mathbf{B}$ trainable. All pre-trained parameters are frozen; only LoRA adapters and the new classification head (totaling $\approx 0.4\text{M}$ parameters) are updated.

Training Details. We use a layered learning rate: 1×10^{-3} for the classification head and 5×10^{-4} for LoRA parameters, optimized with AdamW (batch size 32). Early stopping is based on macro-average F1 on a validation set.

Compiler Integration. A GCC plugin loads the quantized model after Gimple IR generation, predicts flag probabilities, and applies a threshold $\tau = 0.5$ to produce binary decisions. The resulting flags are passed to the compiler backend, enabling seamless integration.

5 Safety Backtracking Mechanism

Deep learning-based flag optimization risks compilation failures or incorrect code due to overly aggressive flags (e.g., `-fsw-fast-math` relaxes IEEE standards). To ensure robustness, we integrate a safety-aware progressive backtracking mechanism into GCC.

Risk Classification. We classify the 85 target flags into three risk levels based on language standards conformance, semantic impact, undefined behavior potential, and historical error data:

- **High:** Violate standards or trigger undefined behavior (e.g., `-ffinite-math-only`, `-fdelete-null-pointer-checks`).
- **Medium:** Generally safe but may fail on rare code patterns (e.g., `-fmodulo-sched`, `-fsched-spec-load`).
- **Low:** Conservative, rarely cause errors (e.g., `-mcpu=sw8a`, `-fgcse-las`).

Backtracking Algorithm. Given the model’s predicted flag vector $\mathbf{F}_{\text{pred}}$ and the risk classification, the algorithm proceeds as follows:

1. Attempt compilation with $\mathbf{F}_{\text{pred}}$; if successful, return the binary.
2. On failure, successively generate new candidates by removing flags of each risk level, first high, then medium, then low, appending them to a retry queue.
3. If all risk-level attempts fail, fall back to the empty flag set, which is equivalent to `-O2`.

This process ensures a valid executable while maximizing performance by retracting only the necessary risky flags. The mechanism adds minimal overhead, averaging 1.33 extra compilations, and makes LLM-driven optimization practical for production use.

6 Experiment and Evaluation

This section presents a systematic evaluation of the proposed SLM-based compiler flag combination optimization method, assessing its effectiveness, generalization capability, and computational efficiency. We conduct three main comparisons: (1) performance against the traditional `-O3` optimization level; (2) performance and reliability against mainstream general-purpose LLMs; and (3) computational resource consumption and inference efficiency compared to LLMs. All experiments were performed on a server equipped with dual SW64 processors, specifically model WX-H8000 running at 2.00 GHz and an NVIDIA A800 80GB GPU, using GCC 12.3.0 to ensure a comprehensive and fair evaluation.

6.1 Experiment

This subsection details the configuration for dataset construction, validation of the safety backtracking mechanism, model performance verification, and efficiency measurements.

Dataset and Benchmark. Training source code is sourced from public datasets and augmented with LLM-generated code, filtered to remove duplicates, low-quality code, and programs that fail to compile or execute. For performance evaluation, we use the SPEC CPU 2017 integer speed benchmark. To ensure accurate timing, each program execution is bound to a dedicated physical core. We perform warm-up runs followed by three timed executions, reporting the

median runtime to minimize noise. Multiple programs are distributed across cores for parallel execution while avoiding resource contention.

Backtracking Mechanism Overhead Analysis. In the worst case, the fallback mechanism algorithm requires $k + 1$ compilation attempts, where $k = 3$ is the number of risk levels. However, since most programs succeed on the initial attempts, the average overhead is much lower. We model the expected overhead probabilistically. Let p_{success} be the probability of success on the initial model prediction. Conditional on initial failure, let p_h, p_m, p_l be the probabilities that the failure is due solely to flags of high, medium, or low risk, respectively. The expected number of extra compilation attempts, $E[\text{extra}]$, is:

$$E[\text{extra}] = (1 - p_{\text{success}}) \cdot [1 \cdot p_h + 2 \cdot p_m(1 - p_h) + 3 \cdot p_l(1 - p_h)(1 - p_m) + 4 \cdot (1 - p_h)(1 - p_m)(1 - p_l)]$$

The coefficients 1, 2, 3 correspond to the additional attempts needed after initial failure when backtracking through high, medium, and low risk levels, respectively; the final 4 accounts for the fallback to the safe baseline optimization level -O2 if all risk-level backtracks fail. Based on failure analysis from the SPEC CPU 2017 integer speed benchmark runs, we estimate $p_{\text{success}} \approx 0.67$ and $p_h \approx 1$, which indicates that most failures are due to high-risk flags, with $p_m \approx p_l \approx 0$. Substituting these values yields $E[\text{extra}] \approx 1.33$. This implies an average increase in compilation time of only about 33% compared to a single successful compilation—an acceptable trade-off for achieving performance gains while ensuring robustness.

Comparison Baselines. We compared the following configurations:

- **GCC -O3:** The standard high-performance optimization level.
- **AutoCFS-SLMs:** Our method, integrating the fine-tuned SLM with safety backtracking in GCC.
- **LLM Baselines:** Performance and reliability comparisons against several widely-used general-purpose LLMs: DeepSeek-V3.2 with 685B parameters, Qwen3-Max with over 1000B parameters, GPT-5.1, and Claude 4.5 Sonnet. These are evaluated via API calls with the same task prompt.

Efficiency Metrics. We measure inference latency, including CPU-based testing on the sw64 processor and GPU-based testing on the NVIDIA A800, along with peak GPU memory footprint, for our SLM, versus a representative 7B-parameter LLM such as CodeLlama-7B.

6.2 Evaluation

Performance Comparison Against -O3 We evaluate AutoCFS-SLMs against the -O3 optimization level baseline on the SPEC CPU 2017 integer speed benchmark. As shown in Figure 4, the configurations recommended by our model achieve performance superior or equal to -O3 across all nine benchmark programs. The geometric mean speedup across the suite is 7.6%, demonstrating the effectiveness of our approach.

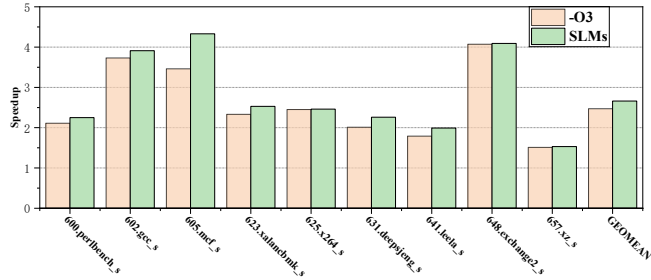


Fig. 4. Speedup achieved by the AutoCFS-SLMs method relative to GCC -O3 across the SPEC CPU 2017 integer speed benchmark. The geometric mean speedup is 7.6%.

Performance Comparison with LLMs To validate the specialized advantage of our model, we compare it against state-of-the-art general-purpose LLMs on the same flag prediction task. We provide each model with an identical prompt: *Recommend the performance-optimal compilation flag combination for a specific benchmark in SPEC CPU 2017 integer speed.* The flags suggested by each model are then used for compilation and performance testing.

Table 1 presents detailed results. Our SLM demonstrates significantly better overall performance than DeepSeek-V3.2, GPT-5.1, and Claude 4.5 Sonnet, and shows performance comparable to or slightly better than Qwen3-Max. A critical observation is that, except for Qwen3-Max and our SLM, the other three models exhibited various compilation or runtime errors. For example, DeepSeek-V3.2 failed to compile `623.xalancbmk_s` due to flag interaction conflicts and crashed at runtime on several other benchmarks. GPT-5.1 produced compilation errors on multiple benchmarks due to model hallucinations. Claude 4.5 Sonnet encountered runtime errors on six of the nine benchmarks.

In contrast, all optimization flags recommended by Qwen3-Max compiled and executed successfully, which we attribute to its massive scale with over 1000B parameters enabling strong generalization. Our SLM, leveraging the safety backtracking mechanism in Section 5, also guaranteed 100% compilation and execution success across all benchmarks. More importantly, since our training dataset inherently contains only flag combinations that compile and run correctly, our model achieves perfect reliability on the SPEC CPU 2017 integer speed benchmark. While some LLMs can achieve excellent performance on individual programs, their instability, including compilation and runtime failures, is unacceptable in practical deployment. Our SLM maintains competitive performance while offering superior reliability and stability.

Computational Efficiency Analysis To quantify the practical advantages of our lightweight approach, we compare the inference overhead of our SLM with a representative larger model, specifically CodeLlama-7B with 7B parameters. Both models were trained and fine-tuned using the same methodology and data.

Table 1. Performance improvement, measured as speedup relative to GCC -O3, across the SPEC CPU 2017 integer speed benchmark for different models. A slash indicates compilation or runtime errors prevented measurement. The geometric mean is calculated only for Qwen3-Max and our SLM, which yielded complete and valid results across all benchmarks.

Model	600	602	605	623	625	631	641	648	657	Geomean Average
DeepSeek-V3.2	1.42%	5.90%	/	/	4.08%	/	4.47%	-47.91%	/	/
Qwen3-Max	5.21%	5.90%	24.28%	6.01%	0.82%	11.94%	13.41%	0.74%	-2.65%	7.03%
GPT-5.1	2.37%	4.56%	/	/	0	9.46%	/	-21.38%	/	/
Claude 4.5 Sonnet	/	4.02%	/	-9.87%	/	/	/	-5.16%	/	/
Our SLMs	6.64%	4.83%	25.14%	8.58%	0.41%	12.44%	11.17%	0.49%	1.32%	7.65%

On the SW64 platform, which lacks CUDA support, we performed CPU-based inference. The inference time for CodeLlama-7B, at 29 minutes and 11 seconds, was 48.1 times that of CodeBERT, which took 36.38 seconds. On the NVIDIA A800 GPU, the peak GPU memory footprint of CodeLlama-7B, reaching 11.04 GiB, was 25.4 times that of CodeBERT, which used only 0.43 GiB. These results demonstrate that SLMs make llm-assisted optimization feasible on resource-constrained edge devices and specialized processors without relying on high-end GPU.

We observe a clear diminishing returns effect: increasing model parameters from 0.11B to 7B, which represents approximately a 63.6-fold increase, yields only marginal performance gains. Specifically, our SLM achieves a geometric mean performance improvement of 7.65%, compared to Qwen3-Max’s 7.03% on the SPEC CPU 2017 integer speed benchmark. Meanwhile the associated computational cost grows super-linearly. This supports the **right-sizing** principle for llm-driven compiler design: selecting the smallest effective model that matches the task complexity. Compiler flag optimization is a structured pattern-matching task with a limited output space, not an open-ended generation problem. SLMs excel at such tasks given sufficient supervised data.

Our research validates that for the specific domain of compiler flag combination optimization, a small, specialized model, specifically CodeBERT with 0.11B parameters, achieves a 48-fold speedup and 25-fold memory efficiency improvement over a larger general-purpose model, specifically CodeLlama-7B with 7B parameters, while maintaining competitive performance. This finding provides strong justification for deploying task-adapted SLMs in practical compilation environments—larger is not always better, and SLMs offer an optimal balance between performance, efficiency, and practicality.

7 Conclusion

This paper investigates the application of SLMs to the compiler flag combination optimization and presents the AutoCFS-SLMs framework. Our work makes three primary contributions. First, we developed an automated pipeline

for constructing a high-quality dataset for learning-based compiler optimization. This tool integrates automatic code acquisition, quality filtering, optimization flag search via an adaptive genetic algorithm, Gimple IR extraction, and the generation of $\langle \textit{Gimple IR}, \textit{Compiler Flags} \rangle$ pairs. This pipeline significantly reduces the data preparation burden for LLM-assisted optimization. We constructed a first-of-its-kind, large-scale compilation flag optimization dataset $\mathcal{D} = (G_i, F_i, P_i)$ for the SW64 processor, where G_i is the Gimple IR, F_i is a near-optimal flag combination, and P_i is the corresponding performance metric. This dataset serves as a valuable resource for future research. Second, we demonstrated the effective specialization of a general-purpose code model for this task. Through domain-adaptive pre-training on Gimple IR, vocabulary expansion, and parameter-efficient fine-tuning, we successfully adapted the CodeBERT model with 110M parameters into a compiler flag combination optimization expert. While achieving a geometric mean performance improvement of 7.6% over the -O3 optimization level baseline on the SPEC CPU 2017 integer speed benchmark, this small language model delivers a 48-fold speedup in inference and a 25-fold reduction in memory footprint compared to a 7B-parameter LLM, validating the **right-sizing** principle for practical deployment. Third, we designed and integrated a safety backtracking mechanism into GCC to ensure robustness. This mechanism classifies optimization flags by risk level and employs a progressive fallback strategy during compilation. It guarantees functional correctness by automatically retracting risky flags upon failure, while still pursuing performance gains. This integration makes llm-driven optimization practical for production environments with stringent reliability requirements. In summary, the AutoCFS-SLMs framework provides an efficient, reliable, and practical pathway for integrating language model intelligence into industrial compilers. It bridges the gap between the promising capabilities of learning-based methods and the stringent demands of real-world compilation toolchains.

References

1. Triantafyllis, S., Vachharajani, M., Vachharajani, N., August, D.I.: Compiler optimization-space exploration. In: International Symposium on Code Generation and Optimization, 2003. CGO 2003. pp. 204–215 (2003)
2. Ansel, J., Kamil, S., Veeramachaneni, K., Ragan-Kelley, J., Bosboom, J., O’Reilly, U.M., Amarasinghe, S.: Opentuner: An extensible framework for program autotuning. In: Proceedings of the 23rd international conference on Parallel architectures and compilation. pp. 303–316 (2014)
3. Almakki, M., Izzeldin, A., Huang, Q., Ali, A.H., Cummins, C.: Autophase v2: Towards function level phase ordering optimization. In: Machine Learning for Computer Architecture and Systems 2022 (2022)
4. Trofin, M., Qian, Y., Brevdo, E., Lin, Z., Choromanski, K., Li, D.: Mlgo: a machine learning guided compiler optimizations framework. arXiv preprint arXiv:2101.04808 (2021)
5. Pan, H., Wei, Y., Xing, M., Wu, Y., Zhao, C.: Towards efficient compiler auto-tuning: Leveraging synergistic search spaces. In: Proceedings of the 23rd

- ACM/IEEE International Symposium on Code Generation and Optimization. pp. 614–627 (2025)
6. Deng, C., Wu, J., Feng, N., Wang, J., Long, M.: Compilerdream: Learning a compiler world model for general code optimization. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2. pp. 486–497 (2025)
 7. Cummins, C., Seeker, V., Grubisic, D., Roziere, B., Gehring, J., Synnaeve, G., Leather, H.: Llm compiler: Foundation language models for compiler optimization. In: Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction. pp. 141–153 (2025)
 8. Zhang, H., Gao, S., Liu, Y., Xing, M., Wu, Y., Zhao, C.: Exploring the feasibility of end-to-end large language model as a compiler. arXiv preprint arXiv:2511.04132 (2025)
 9. Zhang, S., Zhao, J., Xia, C., Wang, Z., Chen, Y., Cui, H.: Introducing compiler semantics into large language models as programming language translators: A case study of c to x86 assembly. In: Findings of the Association for Computational Linguistics: EMNLP 2024. pp. 996–1011 (2024)
 10. Pan, H., Lin, H., Luo, H., Liu, Y., Yao, K., Zhang, L., Xing, M., Wu, Y.: Compiler-r1: Towards agentic compiler auto-tuning with reinforcement learning. arXiv preprint arXiv:2506.15701 (2025)
 11. Almagor, L., Cooper, K.D., Grosul, A., Harvey, T.J., Reeves, S.W., Subramanian, D., Torczon, L., Waterman, T.: Finding effective compilation sequences. ACM SIGPLAN Notices **39**(7), 231–239 (2004)
 12. Mohammed, D., Alwan, E.H., Fanfakh, A.: Compiler sequence optimization using machine learning prediction method. Fusion: Practice and Applications pp. 304–320 (2025)
 13. Grubisic, D., Cummins, C., Seeker, V., Leather, H.: Compiler generated feedback for large language models. arXiv preprint arXiv:2403.14714 (2024)
 14. Cummins, C., Seeker, V., Grubisic, D., Elhoushi, M., Liang, Y., Roziere, B., Gehring, J., Gloeckle, F., Hazelwood, K., Synnaeve, G., et al.: Large language models for compiler optimization. arXiv preprint arXiv:2309.07062 (2023)
 15. Jiang, H., Zhu, J., Wan, Y., Fang, B., Zhang, H., Jin, R., Guan, Q.: Can large language models understand intermediate representations in compilers? arXiv preprint arXiv:2502.06854 (2025)
 16. Sadman, Z., Qasem, A.: Llms in compiler optimization: Challenges and future direction. IEEE Pulse **16**(1), 35–37 (2025)
 17. Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al.: Codebert: A pre-trained model for programming and natural languages. In: Findings of the association for computational linguistics: EMNLP 2020. pp. 1536–1547 (2020)